Robotics Engineering 3001 WPI Final Report *

Mark Kheyfets

***Abstract*— This electronic document is the final report in Robotics Engineering 3001 for Group 20. The final project surrounds a 3 DOF Robotic arm using camera vision for object retrieval. Trajectories are mapped using forward kinematics and reverse kinematics. Camera vision with code adjusted mapping for item detection. Final Project surrounds a pick and place based system, retrieving objects of different colors off a plane and placing them where designated.**

I. Introduction

The fifth and final lab of United Robotics III: Manipulation is a culmination of lessons taught in lectures and capabilities developed for a Dynamixel robotic a rm throughout previous labs. This lab combines previous work done with robot communication, joint and task space control, forwards kinematics, inverse kinematics, and trajectory planning.

Additiona lly, this lab introduces camera vision to robotic manipulation. Using a fisheye camera, colored balls are to be detected by the camera, picked up, and placed in a designated spot by color. New concepts, such as camera calibration, color detection and object detection a re implemented to complete these tasks.

This lab demonstrates how connecting perception and manipulation a llows for an autonomous sorting system. By combining visual detection and kinematic modeling a robotic a rm can perform tasks that require adaptive decisions. This reflects many real-world applications where robots rely on sensors to interact with the world.

Objectives to complete for this lab include understanding how to use and calibrate computer vision systems, convert camera frame objects to robot coordinate frame, control robot end effector, automatically identify and reloca te objects, and write a report.

II. Background information

A. Background

Robotic manipulators are widely used in manufacturing and automation systems. A common example of this is a pick and place system as shown in this lab report, where a robotic a rm detects objects in its environment and moves them to their destination. These systems rely on forward and inverse kinematics, computer vision systems, and to accurately move the system.

To precisely control a robotic arm a relation between a robot's task space and joint space must be made. Forwa rd kinematics can solve for the position and orientation of an end effector with a given set of joint angles. However, in contrast to forward kinematics, inverse kinematics solves for the angles necessary to reach a desired end effector position. Using these kinematic systems together allows for precise movement.

Computer vision is what allows the robot to read and perceive its environment. Photos or videos can be taken by a camera enabling a robot to detect object's position, color, or sha pe. For camera vision to work, camera calibration is necessary. Intrinsic calibration determines the settings of the camera, and Extrinsic calibration helps to establish a spatial rela tionship between the camera fra me and a coordina te plane. The exact process of our calibration is described in the methodology section this report.

Quintic trajectory generation is used in a robot to produce smooth and viable movement between two points. This is accomplished using a fifth-order polynomial to make the position of an end effector or joint as a function of time. With quintic trajectories position, velocity, and acceleration of the beginning and end of the movement can be specified. Using these trajectories also eases the strain on physical components of the robot.

These listed concepts sit at the core of this project; by integra ting them all together it allows for a system that can autonomously detect objects and execute pick and place operations.

B. Motivation

Automa tion has become central to increasing production in factories and manufacturing plants. Tasks like packaging, sorting and material handling are simple and repetitive, making them perfect candidates for automation. A common example of this is a pick and place system, identifying an object, gra sping it, and placing it in a new location. This project allows an opportunity to explore several fundamental concepts rela ted to automation such as kinematics, computer vision, and trajectory planning into a simple system.

III. Methodology

A. Camera Calibration

During the first part of the lab, the main objective included establishing the working connection between MATLAB and USB fisheye camera mounted above the robot workspa ce. The camera was successfully accessed through MATLAB USB camera toolbox, and a live preview was opened to verify that the correct web camera wa s selected and that the checkerboard workspa ce wa s fully visible. This step was important since all further image processing and object detection is relying on a consistent camera view of the board and robot workspace.

After confirming camera's orientation, calibration of the fisheye camera was performed by MATLAB's Camera Calibrator app. Since the insta lled camera uses a fisheye lens, the fisheye and high distortion settings were selected rather than the standa rd configuration. A series of checkerboard ima ges was taken from different angles and orientations in order to allow MATLAB to estimate the internal parameters

of the camera a nd lens distortion. The checkerboard used for calibration had a square size of 25 mm, which was entered into the ca libra tion app so that ima ge could be associated with the real world.

During calibration, ima ges with poor pattern detection or inconsistent axis a lignment were removed. This action improved the qua lity of the pattern detection and parameter estimation. In total, 27 images were kept for the final calibration and the mean pixel error was estimated to be 0.68 pixels, which is below 1 and therefore indica tes a via ble calibration for future measurements.

The final calibration parameters were exported as a MATLAB script so they could be reused by the provided Camera class. This allowed ImageProcessor object to load the calibration parameters each time it was initialized. These steps were necessary because later tasks depended on the camera's properties and distortion correction.

After the calibration wa s completed, the next step was registering the camera image to the robot coordinate system. The purpose of this wa s to transform a selected point in ima ge pixel coordina tes into a usable position in the robot base frame.

B. Camera-Robot Registration

The extrinsic rela tionship between the camera and the checkerboard was obtained by the camera pose estimation functions provided in the Camera class. This returned the rota tion matrix and translation vector describing the pose of the checkerboard relative to the camera . Using these values together with the camera intrinsics, MATLAB's pointsToWorld() function was used to convert selected image points (u, v) into pla nar (x, y) coordinates in the checkerboard reference frame. This step mapped a pixel loca tion in the ima ge to a real loca tion on the board surfa ce in millimeters.

Once the point is expressed in the checkerboard frame, it wa s transformed into robot base frame using a homogeneous transformation matrix determined by inspection of the physical arm. In our implementation, this checkerboard-to-robot transformation was defined as

$$T_0^{board} = \begin{matrix} 0 & 1 & 0 & 89 \\ 1 & 0 & 0 & -112.5 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

Figure 1. Transformation Matrix for board to robot frames

This matrix considers both the translation and rota tion between the checkerboard coordinate frame and the robot base frame. After pointsToWorld() returned the checkerboard coordinates, the point was written as (x, y) and multiplied by this transformation matrix to obtain the position in the robot frame: $[x\ y\ 0\ 1]^T$.

This functionality was then implemented in the ima ge_to_robot() function of the ImageProcessor cla ss. This function first retrieves the camera intrinsics, rotation matrix and translation vector. It then calls pointsToWorld() to determine checkerboard coordinates, constructs o point vector and multiplies it by the checkerboard-to-robot transform matrix to compute the robot frame position. The resulting coordina tes were displayed to verify that chosen checkerboard points mapped to expected robot space loca tions within a reasonable threshold.

C. Object Detection in Image Processing Pipeline

In order for the robotic arm to be able to pick up the balls, it must be able to know where the balls are. This project limited us to one sensor that wa s capable of giving us feedback on the balls position. A singular camera already mounted to the frame surrounding the checkerboard and arm wa s the only sensor required to detect the balls. Matlab has severa l high-level functions for ima ge detection of checkerboards that enabled this pipeline.

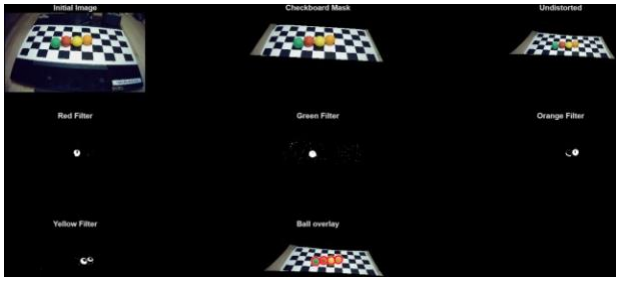


Figure 2. Complete image processing pipeline at each step.

A singular image taken of the board's sta te is sent through severa l functions to filter and ana lyze the color formation for identifying the location, size, and color of the balls. The USB Camera used as a sensor genera ted RGB ima ges with a resolution of 1080p which is stored as an 1080x1920x3 a rray of uint8. The three dimensions of the matrix a re the red, blue, and green color a rrays that each have values of 0-255 across the 1920 horizontal and 1080 vertical pixels. By sepa rating and filtering each color band into a mask only certain colors will be selected.

Before the color bands can be sepa rated it is importa nt that only the useful data in the ima ge is used. To prevent balls that are within the view of the camera but off of the board from being used, the image is ma sked to only the checkerboard region. Matlab has several tools that make this easy and in this case the Lida r toolbox was used. It gave access to the estimateCheckerboardCorners3d() and projectLidarPointsOnImage() functions that can consistently get the edges of the checkerboard and include margins for it. The points gained from those two functions are then given to poly2mask() to get the checkerboard region mask. Actually applying this mask to the image is done with bsxfun().

Every color pixel is stored as a combination of three 0-255 ints for red, green, and blue. The camera does not see the ba lls as solid singula r colors as brightness and lighting mean that 'orange' is actually a range of mixes for red, green, and blue. The Matlab Color Threshold app wa s used to analyze ima ges of each colored ba ll that were then saved

as low and high thresholds of each RGB band per color. Looping for each color has the three color bands broken up and compared against their corresponding thresholds. The three binary comparison bands are then combined with bitwise *and* for a black and white picture that highlights only the designated color.

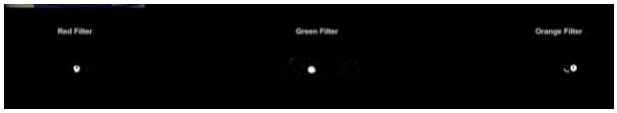


Figure 3. Red, Green, and Orange filters isolated.

This is the best time in the processing pipeline to undistort the image. The camera used has a fisheye lens and thus all images taken with it will have serious warping effects present. With the camera calibration, sufficient data was gained to linearize the image so that distances within it are closer to proportion and kept even throughout with undistortFisheyeImage(). This enables an accurate transformation from pixel coordinates to the checkerboard and then robot frames. When the image is undistorted it no longer has even and consistent dimensions which increases the challenge of applying layered filters, so this is applied afterwards to the filters themselves.

Several functions are then applied to the filtered images to find large, contiguous centroids. bwconncomp() and regionprops() analyze the image to find connected regions and are then combined with a pixel size selection in cc2bw(). To identify large pixel masses of the correct color. The centroid properties are then identified through regionprops() for the pixel coordinate and axes lengths. During debugging this is then displayed onto the undistorted masked image to mark all found balls. This information is returned along with the color filtered applied at the time. At the end of this image processing the location and color of all found balls are the result that get used to coordinate movement later on.

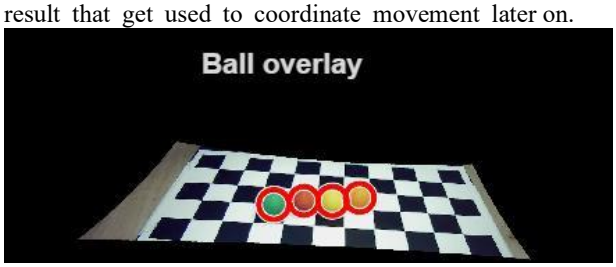


Figure 4. Highlight of balls saved from individual filters

D. Object Localization

An issue arises when trying to pick up balls using only the detect_centroids function. The detection function tries to find the center of ball in x-y coordinates in the checkerboard plane. However, the ball has volume as a sphere and is not flat with the surface of the checkerboard. As a result, an additional function needs to be implemented to ensure the center of a detected ball is accurately represented by checkerboard coordinates.

To resolve this, the exact center of the balls needs to be projected on the x-y checkerboard axis. This can be done using trigonometry and known measurements.

H represents the height difference between the base of the checkerboard to the camera frame. D represents the distance between the origin of the checkerboard frame and the camera frame in terms of the checkerboard y-axis.

The ball's center must be calculated from the visual image location onto the real location on the checkerboard plane. The camera identifies the center of the visible circular outline of the ball, which corresponds to the center of the sphere in three-dimensional space rather than the point where the ball touches the checkerboard surface. Because the camera is positioned at a height **h** above the checkerboard and offset by distance **D** along the checkerboard y-axis, the detected point forms a triangle between the camera frame, the sphere center, and the checkerboard plane. Using these known relationships, trigonometric projection can be applied to determine where the vertical line from the sphere's center intersects the checkerboard plane. This effectively shifts the detected x and y coordinates inward toward the camera origin by an amount dependent on the ball radius and the viewing angle. By applying these methods, the resulting coordinates correspond to the true physical location of the ball on the checkerboard surface, allowing the robot manipulator to accurately plan a grasp trajectory to the correct physical position.

E. Robot Movement

To have our Dynamixel arm move individually from camera input, a separate "pick_up_ball" function was created. The purpose of this function is to move to pick up a ball and drop it off in a specified bin.

This function has two required parameters and three optional parameters. The first parameter is the position of the target ball on the x-y plane of the robot frame. This is the position the arm moves to when attempting to pick up a ball. The second parameter is the color bin the arm should deposit the target ball into. This is entered as a string and has predetermined x-y coordinates. There are five sets of drop off coordinates, for red, orange, yellow, green, and gray colored balls.

Having these two parameters allows this function to run entirely independently from computer vision. With a target pickup position and color bin to drop off at, the arm can take position and color data of a ball and complete the rest of the process with the function.

Additionally, three optional argument exist for advanced control within the nvargs parameter: z_offset, z_ball, and velocity. These default to 50 (mm), 15 (mm), and 150 (mm/s) respectively. The z_offset argument is the height the end effector will be at when transporting colored balls across the checkerboard. Similarly, z_ball will be the height of the end effector when picking up a ball. Velocity is the average velocity the arm will be traveling when moving between all points throughout the function. We decided to use velocity as a parameter to easily be able to adjust arm movement speed across the function.

The function works by first identifying the color input and setting the respective ta rget drop off coordinates. Afterwa rds, six high-level steps will be followed: move the a rm above the ta rget ba ll, lower the a rm and grab the ba ll, lift the arm to the offset height, move the arm to the target drop off coordinates, lower the ball and release, and lift the arm to the offset height. This is a loop that can go from end to end without issue, allowing the arm to pick up multiple balls in succession.

Within each of the six steps, the function calculates or a lters the ta rget position for the a rm to move to. A helper function, “calculate_move_time,” takes the distance between the starting coordinates and ending coordinates and divides it by the velocity argument. This outputs a length of time the a rm will need to take to reach its ta rget position.

Movement trajectories are calculated using the “blocking_ts_move” function. This is a helper function used to easily use inverse kinematics and trajectory planning across a specified period. It uses pa ra meters “pos” and “nvargs,” which has the arguments “time” and “mode.” The parameter "pos" is the target position for the arm to move to in a 1x4 matrix conta ining target x, y, z, and a lpha va lues. The arguments “time” and “mode” set the arm movement period and whether to use cubic or quintic trajectory planning, respectively.

The function first checks the mode argument, and depending on if the argument holds “cubic” or “quintic,” a cubic or quintic trajectory will be calculated. Datapoints for the x, y, z, and alpha positions will be calculated and stored. These values will then be used to calculate inverse kinematics, which will then direct the arm to move to sepa rate positions. Positions are written evenly across the specified period. Updates happen as a matter of updates per second, since issues in previous labs arose when update speeds were too high. This value is stored as a variable and a llows quick changes to update speeds without affecting robot performa nce. Writing to the a rm will use the helper function “blocking_js_move.”

The function “blocking_js_move” is a very simple function that takes “qpos” and “nvargs” parameters, with a time argument. “qpos” is the joint angles for the arm to move to, and “time” is the time for the arm to move to the position. This uses “interpolate_jp” from a previous lab to move the a rm over a specified time in joint space. “blocking_js_move” is not used for large movements in this lab, and is instead used for small steps in the “blocking_ts_move” function. We decided to do this since it a llows inverse kinema tics outputs to be directly input into the helper function, allowing for easy movement instructions.

F. Automated Object Sorting

Autonomous sorting wa s achieved by combining object detection and color detection. Images were captured then processed with an object detection function to find all objects of interest. This process involved segmenting the ima ge and identifying circula r blobs corresponding to the balls placed on the checkerboard surface. Once detected, the centroid of each object was calculated in image coordinates, providing the approximate location of each ball within the camera frame.

After the objects were detected, a color detection step wa s performed to determine the classification of each ball. Color filtering techniques were applied to the image to isolate specific color ranges corresponding to the balls used in the experiment. Each detected object's pixel values were ana lyzed within a sma ll region surrounding its centroid, a llowing the system to determine whether the ba ll was red, blue, green, or another predefined color category. Based on this classification, each ball was assigned a specific target loca tion within the sorting area.

Once both the position and color of each ball were identified, the centroid coordina tes were converted from ima ge space to the checkerboard world coordina te frame using the camera calibration and transformation functions developed earlier in the lab. These coordinates were then used by the robot controller to generate the appropriate motion commands. The robotic arm moved to a position above the detected object, grasped the ba ll, and transported it to the corresponding drop-off location determined by its color classification. This process was repeated iteratively until all detected objects were sorted into their designated loca tions.

IV. RESULTS

A. Camera Calibration

The camera was successfully connected to the MATLAB and integrated with the provided project framework. A live preview confirmed that the camera had a clear view of the checkerboard and that the full usable region for ball detection wa s visible. This verified that the hardware setup wa s viable for image acquisition and later computer vision steps.

For the ca libration, multiple checkerboard images were collected at different angles and positions to account for distortion introduced by the fisheye lens. After reviewing the calibration set. Images with incomplete pattern detection or misa ligned axis were excluded. A total of 27 images were kept for the final process.

The final calibration produced a mean error of 0.68 pixels. This result shows that the estimated camera model closely matched the observed pattern locations. Since the mean error is below 1 pixel is considered acceptable for this type of application, the calibration was determined to be accurate enough for object localization. The distortion correction and intrinsic model provided a reliable basis for converting ima ge coordina tes into physica l coordina tes la ter in the project.

B. Camera-Robot Registration

After intrinsic calibration, the transformation from image coordinates to robot coordinates was tested using selected points on the checkerboard. Pixel locations were chosen from the camera image and converted to checkerboard coordinates using MATLAB’s pointsToWorld() function together with the calibrated intrinsics and the pose of the checkerboard. These checkerboard-frame coordinates were

then transformed into the robot base frame using the manually determined homogeneous transformation matrix.

The registration process produced robot-frame coordinates that were consistent with the expected physical locations of points on the checkerboard. Known points selected at checkerboard intersections mapped to reasonable positions in the robot base frame, confirming that the axis directions, translation offsets, and frame relationships had been defined correctly. This demonstrated that the transformation pipeline from image space to checkerboard space to robot space was functioning as intended.

In practice, this meant that a point selected in the image could be converted into a real position measured in millimeters relative to the robot base. This was an important result because it established that the vision system could provide meaningful spatial targets for the robot. Without this registration step, detected ball centroids would remain only as pixel coordinates and could not be used for motion planning.

V. Discussion

The process of learning during this lab meant that sections went through several iterations from initial plans then morphing as errors were found. In the end with the time constrain this meant that artifacts found their way into the code structure and polishing that could've been done later was continuously put off. Software depreciation and incompatibilities greatly hindered the team throughout design and development.

For personal productivity purposes, most teammates were not actively using a Linux dual-boot with designated software loaded onto it – instead doing steps on Windows and Mac machines. At points this was necessary as critical issues emerged with Matlab on Linux having several broken graphics drivers and interfaces. This made it difficult to display the Webcam, calibrate the camera, and interact with selective sections of MatLab's user interface. However, the result of these struggles meant that only a single team member had the current UpToDate Git branch loaded onto a system capable of interfacing with the robotic arm.

Earlier on in this lab the attempted use of a crucial function took significant time to troubleshoot and debug before identifying it as depreciated. Other unreliable portions of the software and learning curves with the camera and its calibration meant that the steps for the second signoff were repeated many times before properly solving the issues.

During the construction of the image processing pipeline an additional Matlab application was used for functions in the Lidar Toolbox. This made it easier to find the checkerboard corners but later played into the software issues experienced on Linux systems. The camera testing and object identification was done while the webcam was attached to a Windows system that had no problem installing the Lidar Toolbox. However, once the move had to be made to incorporating it with the Linux machine that was controlling the arm they found it impossible to use the designed system for identifying checkerboard corners. At this point a quick solution had to be prepared of using predesignated pixel coordinates for the checkerboard corner while another solution could be worked on. This shows the importance of keeping the system baseline and requirements consistent throughout the project so that new dependencies do not cause sudden incompatibilities across the team.

VI. Conclusion

This lab demonstrates the integration of different robotics concepts that all combine to allow for an autonomous system. The use of camera-based object detection allowed the robot to locate objects within its workspace, while coordinate transformations converted the detected image locations into real-world positions that the robot could reach. Additionally, the implementation of color detection enabled the system to classify objects and determine their appropriate sorting locations. Trajectory planning using smooth motion profiles ensured that the robot could move between pick and place positions in a controlled and reliable manner. Adjustments such as correcting the projected ball center improved the accuracy of object localization, allowing the robot to successfully grasp objects despite the limitations of camera perspective.